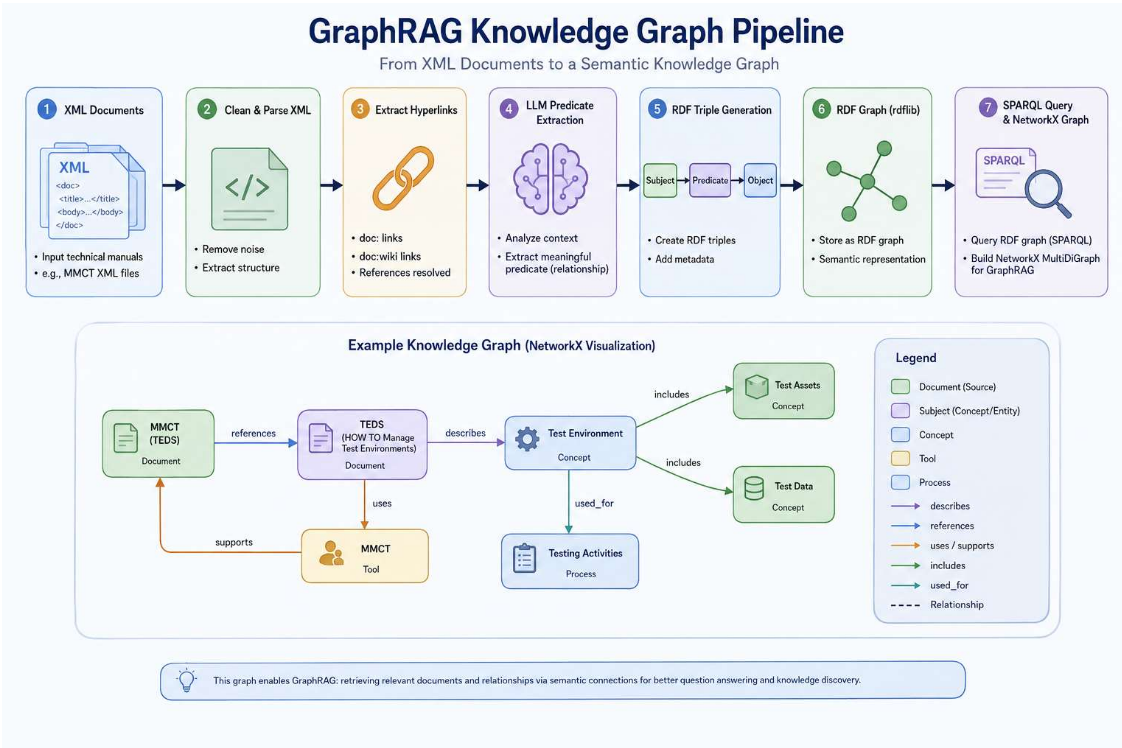
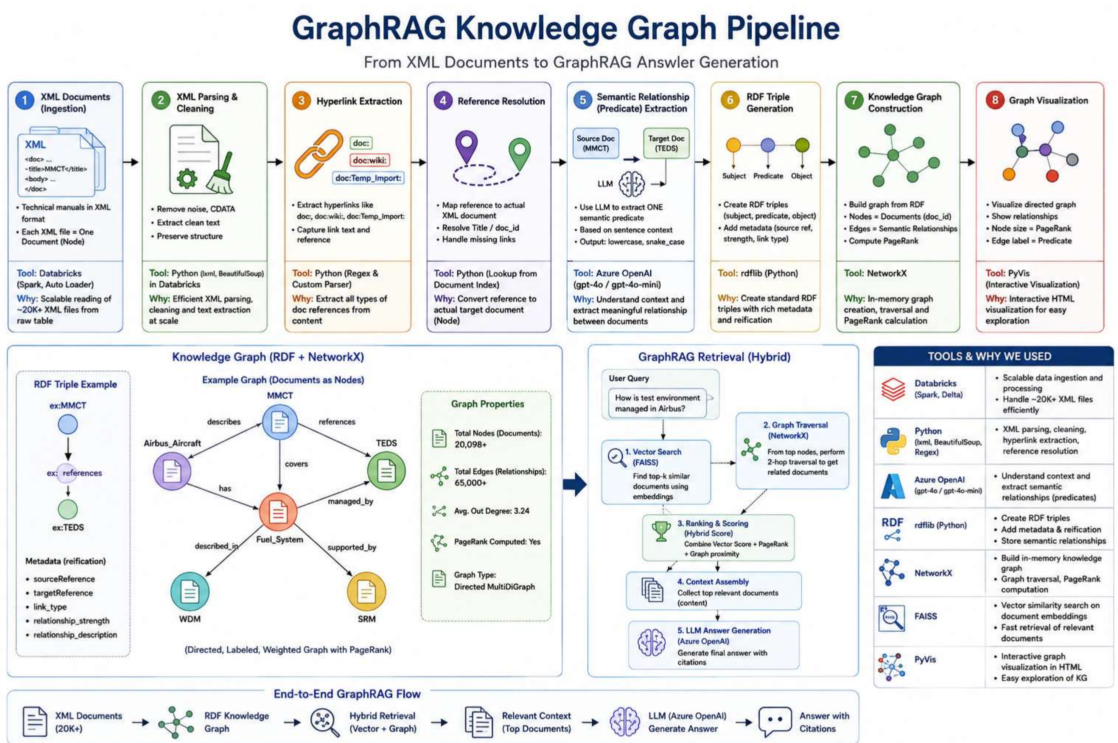
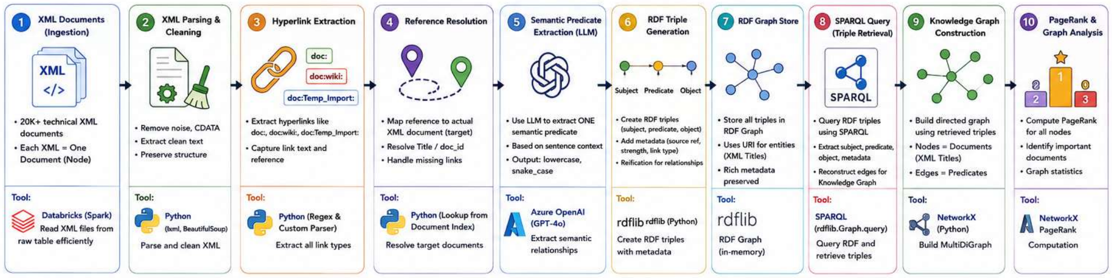
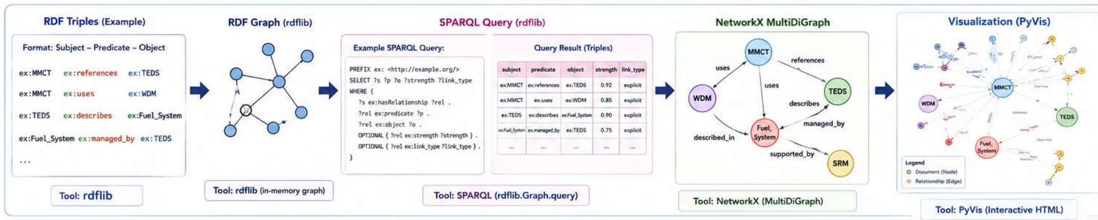

GraphRAG Knowledge Graph Pipeline
From XML Documents to a Semantic Knowledge Graph
1 XML Documents
XML
<doc>
<title>...</title>
<body>...</body>
</doc>
• Input technical manuals
• e.g., MMCT XML files
2 Clean & Parse XML
• Remove noise
• Extract structure
3 Extract Hyperlinks
• doc: links
• doc:wiki links
• References resolved
4 LLM Predicate Extraction
• Analyze context
• Extract meaningful predicate (relationship)
5 RDF Triple Generation
Subject
Predicate
Object
• Create RDF triples
• Add metadata
6 RDF Graph (rdflib)
• Store as RDF graph
• Semantic representation
7 SPARQL Query & NetworkX Graph
SPARQL
• Query RDF graph (SPARQL)
• Build NetworkX MultiDiGraph for GraphRAG
Example Knowledge Graph (NetworkX Visualization)
MMCT (TEDS)
Document
references
TEDS (HOW TO Manage Test Environments)
Document
describes
Test Environment
Concept
includes
Test Assets
Concept
includes
Test Data
Concept
uses
used_for
supports
MMCT
Tool
Testing Activities
Process
Legend
Document (Source)
Subject (Concept/Entity)
Concept
Tool
Process
describes
references
uses / supports
includes
used_for
Relationship
This graph enables GraphRAG: retrieving relevant documents and relationships via semantic connections for better question answering and knowledge discovery.


GraphRAG Knowledge Graph Pipeline
From XML Documents to GraphRAG Answler Generation
1 XML Documents (Ingestion)
XML
<doc> ...
<title>MMCT</title>
<body> ...
</doc>
• Technical manuals in XML format
• Each XML file = One Document (Node)
Tool: Databricks (Spark, Auto Loader)
Why: Scalable reading of ~20K+ XML files from raw table
2 XML Parsing & Cleaning
• Remove noise, CDATA
• Extract clean text
• Preserve structure
Tool: Python (lxml, BeautifulSoup) in Databricks
Why: Efficient XML parsing, cleaning and text extraction at scale
3 Hyperlink Extraction
doc:
doc:wiki:
doc:Temp_Import:
• Extract hyperlinks like doc:, doc:wiki:, doc:Temp_Import:
• Capture link text and reference
Tool: Python (Regex & Custom Parser)
Why: Extract all types of doc references from content
4 Reference Resolution
• Map reference to actual XML document
• Resolve Title / doc_id
• Handle missing links
Tool: Python (Lookup from Document Index)
Why: Convert reference to actual target document (Node)
5 Semantic Relationship (Predicate) Extraction
Source Doc (MMCT)
Target Doc (TEDS)
LLM
• Use LLM to extract ONE semantic predicate
• Based on sentence context
• Output: lowercase, snake_case
Tool: Azure OpenAI (gpt-4o / gpt-4o-mini)
Why: Understand context and extract meaningful relationship between documents
6 RDF Triple Generation
Subject
Predicate
Object
• Create RDF triples (subject, predicate, object)
• Add metadata (source ref, strength, link type)
Tool: rdflib (Python)
Why: Create standard RDF triples with rich metadata and reification
7 Knowledge Graph Construction
• Build graph from RDF
• Nodes = Documents (doc_id)
• Edges = Semantic Relationships
• Compute PageRank
Tool: NetworkX
Why: In-memory graph creation, traversal and PageRank calculation
8 Graph Visualization
• Visualize directed graph
• Show relationships
• Node size = PageRank
• Edge label = Predicate
Tool: PyVis (Interactive Visualization)
Why: Interactive HTML visualization for easy exploration
Knowledge Graph (RDF + NetworkX)
RDF Triple Example
ex:MMCT
ex: references
ex:TEDS
Metadata (reification)
• sourceReference
• targetReference
• link_type
• relationship_strength
• relationship_description
Example Graph (Documents as Nodes)
MMCT
describes
references
Airbus_Aircraft
TEDS
covers
has
managed_by
Fuel_System
described_in
supported_by
WDM
SRM
(Directed, Labeled, Weighted Graph with PageRank)
Graph Properties
Total Nodes (Documents): 20,098+
Total Edges (Relationships): 65,000+
Avg. Out Degree: 3.24
PageRank Computed: Yes
Graph Type: Directed MultiDiGraph
GraphRAG Retrieval (Hybrid)
User Query
How is test environment managed in Airbus?
1. Vector Search (FAISS)
Find top-k similar documents using embeddings
2. Graph Traversal (NetworkX)
From top nodes, perform 2-hop traversal to get related documents
3. Ranking & Scoring (Hybrid Score)
Combine Vector Score + PageRank + Graph proximity
4. Context Assembly
Collect top relevant documents (content)
5. LLM Answer Generation (Azure OpenAI)
Generate final answer with citations
TOOLS & WHY WE USED
Databricks (Spark, Delta)
• Scalable data ingestion and processing
• Handle ~20K+ XML files efficiently
Python (lxml, BeautifulSoup, Regex)
• XML parsing, cleaning, hyperlink extraction, reference resolution
Azure OpenAI (gpt-4o / gpt-4o-mini)
• Understand context and extract semantic relationships (predicates)
RDF
rdflib (Python)
• Create RDF triples
• Add metadata & reification
• Store semantic relationships
NetworkX
• Build in-memory knowledge graph
• Graph traversal, PageRank computation
FAISS
• Vector similarity search on document embeddings
• Fast retrieval of relevant documents
PyVis
• Interactive graph visualization in HTML
• Easy exploration of KG
End-to-End GraphRAG Flow
XML Documents (20K+)
RDF Knowledge Graph
Hybrid Retrieval (Vector + Graph)
Relevant Context (Top Documents)
LLM (Azure OpenAI) Generate Answer
Answer with Citations

# Knowledge Graph Creation Pipeline (Our Project)

From XML Documents to Knowledge Graph (RDF + NetworkX)




## TOOLS USED & WHY

- **Databricks (Spark)** — Scalable ingestion and processing of 20K+ XML files from raw table
- **Python (lxml, BeautifulSoup, Regex)** — XML parsing, cleaning, noise removal, hyperlink extraction and reference resolution
- **Azure OpenAI (GPT-4o)** — Extract one semantic predicate (relationship) between source and target documents
- **rdflib (Python)** — Create RDF triples, store RDF graph, add metadata and reification
- **SPARQL (rdflib)** — Query RDF graph to retrieve triples and reconstruct edges for the knowledge graph
- **NetworkX (Python)** — Build directed knowledge graph (MultiDiGraph), compute PageRank, graph statistics
- **PyVis** — Interactive visualization of the knowledge graph in HTML for easy exploration

This pipeline builds a high-quality Knowledge Graph from 20K+ XML documents using semantic relationships extracted by LLM, represented in RDF, queried using SPARQL, and visualized as an interactive graph.

## Top box = Ontology (Schema / Plan)

### ENTITY TYPES (CLASSES)

**Document**
Represents a source XML document in the knowledge graph.

**Referenced_Document**
Represents a document that is referenced (target) by another XML document.

### RELATIONSHIP TYPES (PROPERTIES / PREDICATES)

| Predicate | Meaning |
|---|---|
| references | Source document references another document. |
| uses | Source document uses another document. |
| describes | Source document describes or explains another document. |
| managed_by | Source document is managed by another document. |
| supports | Source document supports another document. |
| contains | Source document contains information about another document. |
| implements | Source document implements or is based on another document. |
| related_to | General relationship when no specific relation is identified. |

### DATATYPE PROPERTIES

- **label** : Human-readable title of document
- **sourceReference** : Original reference from source XML
- **targetReference** : Original reference of target XML
- **documentContent** : Full text content of the XML document
- **relationship_description** : Natural language description of the relationship
- **relationship_strength** : Confidence score (0.0 - 1.0)
- **link_type** : Type of hyperlink (doc:, doc:wiki:, doc:Temp_Import:)

## Bottom box = Entities and Relationships (Actual Data)

| Source | Relationship | Target | Details |
|---|---|---|---|
| MMCT (Document) | references | TEDS (Referenced_Document) | Description: "MMCT references TEDS for test environment details." Strength: 0.92 \| Link Type: doc:wiki \| SourceRef: doc:MMCT \| TargetRef: doc:TEDS |
| TEDS (Document) | describes | Fuel_System (Referenced_Document) | Description: "TEDS describes the Fuel System components and architecture." Strength: 0.85 \| Link Type: doc:wiki \| SourceRef: doc:TEDS \| TargetRef: doc:Fuel_System |
| MMCT (Document) | uses | WDM (Referenced_Document) | Description: "MMCT uses WDM for data management." Strength: 0.89 \| Link Type: doc: \| SourceRef: doc:MMCT \| TargetRef: doc:WDM |
| Fuel_System (Document) | managed_by | TEDS (Referenced_Document) | Description: "Fuel System is managed by TEDS." Strength: 0.90 \| Link Type: doc:wiki \| SourceRef: doc:Fuel_System \| TargetRef: doc:TEDS |
| SRM (Document) | supports | MMCT (Referenced_Document) | Description: "SRM supports MMCT processes." Strength: 0.87 \| Link Type: doc:Temp_Import: \| SourceRef: doc:SRM \| TargetRef: doc:MMCT |

## Top box = Ontology (the schema/plan)

**ENTITY TYPE (CLASS)**

**XML_Document**

Represents an XML document from the corpus.

**RELATIONSHIP TYPES (PREDICATES)**

```
XML_Document --references--> XML_Document
XML_Document --describes--> XML_Document
XML_Document --uses--> XML_Document
XML_Document --managed_by--> XML_Document
XML_Document --supports--> XML_Document
XML_Document --contains--> XML_Document
XML_Document --implements--> XML_Document
XML_Document --related_to--> XML_Document
```

**DATATYPE PROPERTIES**

```
label
documentContent
sourceReference
targetReference
relationship_description
relationship_strength
link_type
```

These are types/categories and properties defined in the ontology. "XML_Document" is the category, not an actual document.

## Bottom box = Entities and relationships (the actual data)

**EXAMPLE ENTITIES AND RELATIONSHIPS (ACTUAL DATA)**

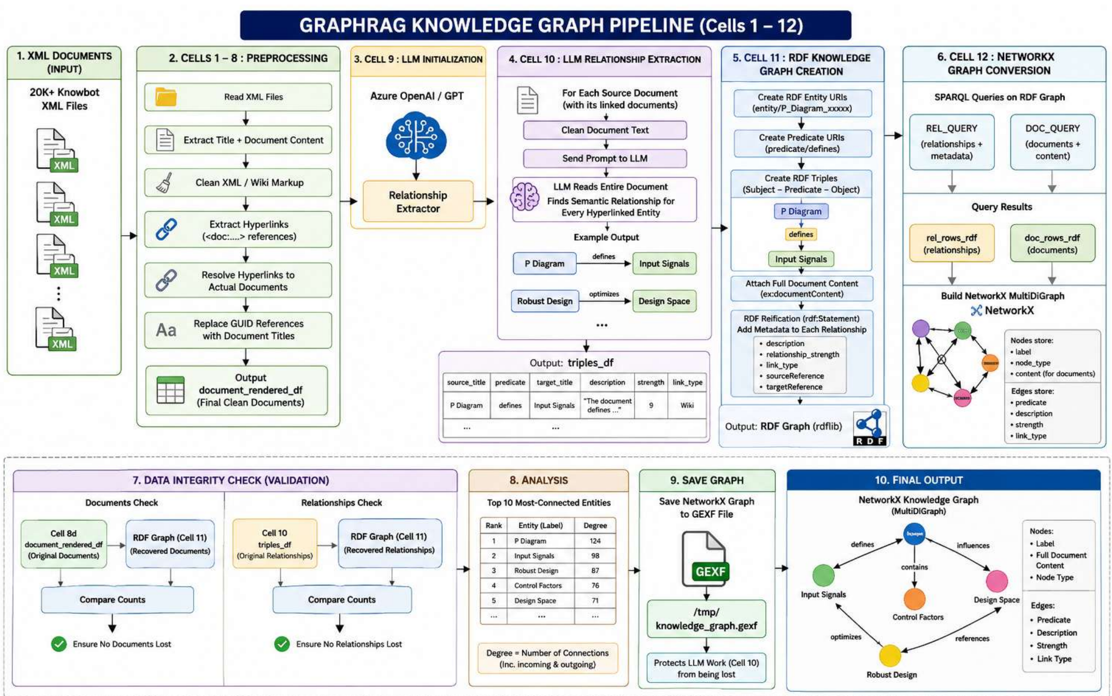
| | | | |
|---|---|---|---|
| MMCT | --references--> | TEDS | **Description:** "MMCT references TEDS for test environment details." **Strength:** 0.92 \| **Link Type:** doc:wiki \| **SourceRef:** doc:MMCT \| **TargetRef:** doc:TEDS |
| TEDS | --describes--> | Fuel_System | **Description:** "TEDS describes the Fuel System components and architecture." **Strength:** 0.85 \| **Link Type:** doc:wiki \| **SourceRef:** doc:TEDS \| **TargetRef:** doc:Fuel_System |
| MMCT | --uses--> | WDM | **Description:** "MMCT uses WDM for data management." **Strength:** 0.89 \| **Link Type:** doc \| **SourceRef:** doc:MMCT \| **TargetRef:** doc:WDM |
| Fuel_System | --managed_by--> | TEDS | **Description:** "Fuel System is managed by TEDS." **Strength:** 0.90 \| **Link Type:** doc:wiki \| **SourceRef:** doc:Fuel_System \| **TargetRef:** doc:TEDS |
| SRM | --supports--> | MMCT | **Description:** "SRM supports MMCT processes." **Strength:** 0.87 \| **Link Type:** doc:Temp_Import \| **SourceRef:** doc:SRM \| **TargetRef:** doc:MMCT |

These are the actual documents (entities) and the relationships extracted and stored in the knowledge graph as RDF triples.

**Reference**

This ontology design and the overall graph-based RAG approach are inspired by:

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Osazuwa Ness, R., & Larson, J. (2025). *From Local to Global: A GraphRAG Approach to Query-Focused Summarization.* Preprint. arXiv:2404.16130.




**SUMMARY:** XML Documents → Preprocess → LLM Extracts Relationships → RDF Knowledge Graph (with metadata & content) → SPARQL Query → NetworkX Graph → Validate → Analyze → Save for GraphRAG Retrieval & Visualization

# GraphRAG Pipeline – RDF Based Knowledge Graph from XML Documents